

Frontend Integration Blueprint

RULE-BASED ANALYTICS & ACADEMIC INTEGRITY MONITORING

ARCHITECTURE TARGET	FRAMEWORK CONTEXT	BACKEND PILLARS	PROJECT DOMAIN
Vibecoding Full-Stack Specification	React / Vite Frontend Engine	Deterministic Rules (No AI)	CodeInsight Platform

1. Design Philosophy & Data Flow

This blueprint maps the frontend integration for the deterministic analytics suite of CodeInsight. All core computations are processed strictly via rule-based metrics on the backend (using compiler errors, Tree-sitter AST nodes, and time-delta telemetry), which means the frontend only needs to consume pre-structured JSON models. This eliminates AI latency, token costs, and non-deterministic variations.

The Vertical Slice Synchronization Model

To prevent the development agent from suffering from backend bias, all workspace additions must enforce a rigid interface pipeline. The data contract must progress linearly from database schemas up to components:

```
[PostgreSQL Database View] → [Express Controller Serialization] → [React API Fetch Hook] → [State Manager] → [UI Presentation View]
```

2. Feature Specifications & Data Contracts

FEATURE 1

Student Micro-Concept Alert Cards

Purpose	Expose instant, granular structural feedback within instructor panels whenever a student encounters explicit anti-patterns or syntax blockages.
Scope	Instructor Dashboard (Live Monitoring Stream)
Business Logic	Triggers when the backend evaluation framework logs a submission containing structural missteps. Aggregates code snippets against explicit AST matcher criteria.

Expected API JSON Payload Structure

```
{  "studentId": "d3b07384-d113-4c4e-a540-5e1a90c128bb",  "studentName": "Rosmer Del Mundo Hizer",  "microConceptDifficulty": "Pointer Dereferencing (Operator Confusion)",  "evidence": "Line 42: int* ptr = &val; std::cout << *ptr;",  "instructorNote": "Student is accessing an address reference with excessive uninitialized assignment syntax patterns."}
```

FEATURE 2 Class-Wide Misconception Report Card

Purpose	Synthesize structural programming patterns across all active student records once an assignment window is closed.
Scope	Instructor Dashboard (Historical Assignment View)
Business Logic	Tallies matching AST failure identifiers across submissions, sorts instances by count, and runs percentage ratios against total class enrollment metadata.

Expected API JSON Payload Structure

```
{
  "mostCommonIssue": "Infinite While-Loop Conditions",
  "affectedCount": 14,
  "affectedPercent": 47,
  "secondIssue": "Off-by-One Array Index Boundaries",
  "secondCount": 6,
  "classSummary": "Nearly half the class failed to implement structural iteration counters, causing backend execution timeouts.",
  "rootCause": "Gap in tracking state modifications manually prior to compiling source trees.",
  "recommendedAction": "Dedicate 5 minutes of next class session to tracing iterative mutations.",
  "beforeAdvancing": "Verify students can cleanly map conditional exit branches."
}
```

FEATURE 3 Longitudinal Student Progress Tracker

Purpose	Track and chart mastery milestones chronologically, helping instructors monitor if specific logic issues settle over continuous work cycles.
Scope	Dual View (Instructor Student Drill-down & Student Self-Monitoring Tab)
Business Logic	Performs sequential groupings of submission evaluation logs by assignment date, mapping the reduction of code complexity and syntax alerts.

Expected API JSON Payload Structure

```
{
  "studentId": "d3b07384-d113-4c4e-a540-5e1a90c128bb",
  "timeline": [
    { "date": "2026-05-10", "masteryScore": 22.5, "flaggedConcepts": ["Syntax Missing Semicolon"] },
    { "date": "2026-05-17", "masteryScore": 58.0, "flaggedConcepts": ["Array Out of Bounds"] },
    { "date": "2026-05-24", "masteryScore": 91.2, "flaggedConcepts": [] }
  ],
  "resolvedTrends": ["Syntax Errors", "Loop Limits"],
  "remainingAreas": ["Dynamic Pointer Allocation Safety"]
}
```

FEATURE 4 Academic Integrity Monitoring System

Purpose	Highlight extreme behavioral flags such as hardcoding literal expected test values or instantly submitting solutions from an empty state.
Scope	Instructor Verification Panel & Roster View
Business Logic	Evaluates source files for literal string output streams matching tests without variable use, or triggers on extreme timing deltas combined with high score status.

Expected API JSON Payload Structure

```
{
  "submissionId": "f78c91a2-11bc-49de-821e-c1a0198fbc10",
  "isHardcoded": true,
  "behavioralAnomaly": true,
  "anomalyWeight": 0.94,
  "details": "Perfect submission executed in 4 seconds following a continuous sequence of compile errors."
}
```

3. Suggested UI Layout Mockups

Live Student Analytics & Integrity Dashboard Interface

CodeInsight :: Instructor Live Monitoring Console

⚠ MICRO-CONCEPT MISCONCEPTION LOGGED

Active Session

Student: Rosmer Del Mundo Hizer

Issue: Pointer Dereferencing (Operator Confusion)

```
Line 42: int* ptr = &val; std::cout << *ptr;
```

 Action Plan: Verify student has explicit understanding of object location bindings before dereferencing storage elements.

 **CRITICAL INTEGRITY ANOMALY
TRIGGERED**

WEIGHT: 94%

Violation: Literal Constant Hardcoding Countermeasure

Telemetry: Source directly outputs structural string literals matching test assertions instead of building target variable manipulation sequences.

Class-Wide Historical Report Panel Interface

CodeInsight :: Class-Wide Metrics Card [Exercise Block #3]

47%

PRIMARY ISSUE CONCENTRATION

14 Students

AFFECTED GROUP TALLY

Most Frequent Failure Mode:

Infinite While-Loop Conditions

Summary Statement:

Nearly half the active workshop cohort missed tracking state updates inside iteration trees, triggering loop termination errors.

 **Probable Instructional Gap:** Manual memory block visualization shortcuts missing during lecture tracking intervals.

 **Prescribed Next Step:** Conduct a 5-minute manual loop configuration breakdown at the next session.

 **Gatekeeper Rule:** Do not deploy nested sorting assignments until loop exits are verified.

4. Step-by-Step Vibecoding Execution Prompt

Agent Operational Instructions:

Copy and paste the code block below directly into your Gemini/Claude CLI window to systematically build the UI components mapped above without introducing regressions into the active code base.

```
[EXECUTE VERTICAL SLICE ARCHITECTURE FOR FRONTEND ANALYTICS]
We are integrating the frontend views for the deterministic, rule-based analytics suite. All
backend services (`microConceptEngine.js`, `astVerifier.js`) are stabilized. You must build out
the client consumption interface now.

Follow this execution loop precisely:
1. File Generation - Client Handler: Create `frontend/src/api/analyticsService.js` to define
Axios wrapper methods handling fetch operations against:
  - GET `/api/analytics/alerts` (Micro-concept flags)
  - GET `/api/analytics/report/:exerciseId` (Class-wide insights)
  - GET `/api/analytics/progress/:studentId` (Longitudinal track data)
  - GET `/api/analytics/integrity` (Anomalous activity metrics)

2. Component Isolation - Presentation Elements: Build modular React components matching our
design specs inside `frontend/src/components/analytics/`:
  - `MicroConceptAlertCard.jsx` (Using background color #ffffbeb, matching left border amber
accent)
  - `ClassMisconceptionReport.jsx` (Displaying breakdown metrics, summaries, and instructional
blockers)
  - `IntegrityMonitoringBanner.jsx` (Displaying deep warning states for hardcoded test
patterns)

3. Dashboard Composition: Integrate these presentation layouts inside `frontend/src/pages/`
```

InstructorDashboard.jsx`. Wire the hooks using proper `useState` and `useEffect` cycles to pull from our endpoints.

Execute code assembly now. Run 'npm test' or verify builds upon completing components to ensure no breakages occur.